7 Reference

FULL

Why these programs? (questions and answers)

Quick version

No time to read it all? This is the short version. The full guide is below.

- Questions and answers on why the garden is built this way. Every answer comes from something that really happened on the old server.
- Ubuntu, not Windows: a Windows update and a frozen Docker Desktop took the old site down. The Linux server next to it never stopped.
- Docker runs each program in its own sealed box, so programs can't mess up Ubuntu or each other.
- Only two ports are open (80 and 443), because every open port is a door someone on the internet can knock on.
- Practise with a `.test` name first: Let's Encrypt allows only a few certificates per name per week.
- Many backups, with copies off the laptop: a backup on the same laptop protects you from mistakes, not from a dead laptop.
- Change one thing at a time, then test. Then you always know what broke it.

This page explains, as questions and answers, why the garden is built the way it is. Every answer comes from something that really happened while the garden was running on the old server. Some of these lessons took hours or days to learn. You get them for free.

Each answer ends with **Read more**, the official page, so you can check it yourself.

The computer

Q: Why Ubuntu, and not Windows? A: On the old setup the website ran inside Docker Desktop on a Windows PC. One night a Windows update restarted the PC by itself at about 2:30 am, and the site went down. Another time Docker Desktop's engine froze: every request got an error, and it wrote no log at all. Both public sites went dark together. The Linux server next to it never stopped. So the garden runs on Linux. Ubuntu **LTS** ("long-term support") gets free safety updates for five years, and most guides on the internet are written for it. Read more: https://ubuntu.com/about/release-cycle

Q: Why an old laptop, and not my desktop? A: A server must stay on all the time. A laptop uses little power, and its battery works as a small backup power supply. The setup keeps it awake with the lid closed. If you would rather use your desktop, a Raspberry Pi or another computer, choose "another computer" in the menu and give its address. Read more: https://releases.ubuntu.com/24.04/ (the tested version; guide 01 explains why)

Q: Why does the laptop need a fixed (reserved) address? A: The router forwards internet visitors to one address. If the laptop gets a new address after a restart, visitors are sent to nobody (or to the wrong computer). Reserving the address in the router stops that. The checklist warns you if it ever changes. Read more: `START-HERE/14-reserve-your-ip.md`

Docker and the programs

Q: What is Docker, and why use it? A: Docker runs each program in its own sealed box, called a container. The box holds everything the program needs. Programs can't mess up Ubuntu or each other, and one file (`docker-compose.yml`) says how they all start. The old server ran many containers this way. The lesson there was to keep only what you use: it once had more than a hundred, and nobody knew what half of them did. Your garden has a handful, and every one is explained. Read more: https://docs.docker.com/get-started/docker-overview/

Q: Why MariaDB? A: The garden was built on MySQL. MariaDB is the free community version made by MySQL's original authors. It understands exactly the same commands. Read more: https://mariadb.org/about/

Q: Why is the website written in Rust? A: It started as PHP. It was moved to Rust page by page, and every page was checked against the old one before the next was started. Rust uses little memory, which matters on an old laptop. It also checks a lot before the program can even be built. For example, if a page uses a field that doesn't exist, the build stops and tells you, instead of the page breaking for visitors. The look, the addresses (even the `.php` ones that printed QR codes still use) and the phone app all stayed the same. Read more: https://www.rust-lang.org/learn, `docs/rust-port/CODE-TOUR.md`

The front door

Q: Why Caddy? A: Caddy gets the HTTPS padlock certificate by itself and renews it by itself. Its settings file is short and easy to read. Apache and nginx can do the same job, but need more steps. (`lab/06` installs Apache next to it, just for learning.) Your Caddy runs directly on Ubuntu, not in Docker, which avoids a trap from the old server: Caddy in a container kept reading an old copy of its settings file after the file was replaced. Read more: https://caddyserver.com/docs/

Q: Why only two ports open? A: Every open port is a door someone on the internet can knock on. Ports 80 and 443 go to Caddy, and Caddy decides what each visit may reach. Everything else (website, database, wiki, tools) listens only on the laptop itself. The old server had a "no new open ports" rule for the same reason. Read more: https://ubuntu.com/server/docs/how-to/security/firewalls/

Q: Why does Caddy delete headers like X-Auth-Email from visitors? A: The garden tools trust a header that says who you are, but only when the sign-in page put it there. A visitor could try to send that header themselves to pretend to be you. Caddy throws away anything like it before passing the visit on. This is copied from the old server's settings. Read more: https://caddyserver.com/docs/caddyfile/directives/request_header

Q: Why a separate sign-in page (Authelia)? A: The garden tools have no login of their own. On the old server a sign-in gate stood in front of them, and Authelia does the same job for you. It is a well-known open-source program with good documentation, so you can always look things up. It also slows down password guessing: 5 wrong tries in 2 minutes locks the name for 10 minutes. The website keeps its own login for editing plants. Read more: https://www.authelia.com/overview/prologue/introduction/

Q: Why No-IP, and why do I confirm the name every month? A: Your home internet address can change. The No-IP updater tells No-IP the new address within minutes. `junsgarden.zapto.org` is a free No-IP name, and free names must be confirmed every 30 days (No-IP emails you a link). If you ever want to skip that, a paid No-IP plan or your own domain (about $12 a year) does it. Read more: https://www.noip.com/support/knowledgebase/

Q: Why practise with a ".test" name first? A: The free certificate service (Let's Encrypt) allows only a few certificates per name per week. A mistake repeated a few times can lock you out for days. A `.test` name uses a practice certificate made by Caddy itself, so you can make every mistake for free. Read more: https://letsencrypt.org/docs/rate-limits/

Keeping it safe

Q: Why is every password in one `.env` file I never have to open? A: Passwords in code end up shared by accident. On the old server a password was once shown on screen by mistake and had to be changed everywhere. So passwords live only in `.env`, only you can read it, and the setup makes them without ever showing them. Settings (menu `8`) changes them for you. If `.env` is ever shown to anybody, use Settings, "Renew the internal database passwords". Read more: https://owasp.org/www-project-top-ten/

Q: Why so many backups, and why copy them off the laptop? A: On the old setup 860 GB of photos had only one copy, and a storage box had a failing disk at the same time. That is how people lose everything. A backup on the same laptop protects you from mistakes, not from a dead laptop. So: nightly backups, plus a copy on a USB stick or another

computer now and then. Read more: https://www.backblaze.com/blog/the-3-2-1-backup-strategy/

Q: Why does every backup have a MANIFEST and SHA256SUMS? A: The manifest says what is inside. The SHA256 fingerprints prove nothing is missing or damaged. A restore checks them before it touches anything. Read more: https://www.gnu.org/software/coreutils/manual/html_node/sha2-utilities.html

Q: Why free AI only, and no paid keys? A: Paid keys cost money every time something calls them, and a mistake can cost a lot. The garden uses free tiers: Google's Gemini free key and OpenRouter's free models. Free services are sometimes busy, so the garden tools put work in a queue and try again later instead of failing. Local AI on a strong home computer is the private option (guide 15). Read more: https://ai.google.dev/gemini-api/docs/rate-limits

Working on it

Q: Why "one change at a time, then test"? A: Once, a single comment put in the wrong place (inside a database command) quietly broke every login. Only testing caught it. The Rust move was done one page at a time, each with a test site and a way back. If you change one thing and test, you always know what broke it. Read more: `START-HERE/11-maintain-your-site.md`

Q: Why git? A: git keeps every saved version. `git diff` shows exactly what you changed, and `git checkout -- FILE` undoes it. It is the safety net under every edit. Read more: https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F

Q: Why edit files on Ubuntu, not Windows? A: Windows and Linux end lines in text files differently. On the old setup, Windows' git quietly changed some files when saving them, and even broke some PDFs. Shell scripts with Windows line endings fail on Linux. Edit on the laptop, or set git to leave line endings alone. Read more: https://docs.github.com/en/get-started/getting-started-with-git/configuring-git-to-handle-line-endings

Q: Why one goal at a time? A: On the old server, days went into exciting extra features that were then dropped. Finishing one thing that works beats starting five. Keep a "later" list for new ideas.

Q: What if I get stuck? A: Menu `H` makes a help file: what happened and where the install stopped, with no passwords in it. Read it, then paste it into Gemini or show it to someone you trust. The checklist (menu `5`) tells you what to fix first.
